

Project Title	OCD Patient Dataset: Demographics & Clinical Data (regulatory affairs)
Tools	Jupyter Notebook and VS code
Technologies	Machine learning , Python,Excel
Domain	Data Science
Project Difficulty level	Intermediate

PHASE 01: Data Loading and Understanding

Objective: The primary goal of this phase is to load the dataset, inspect its structure, and prepare it for further preprocessing by renaming columns into consistent, machine-friendly names.

a) Import Libraries : Import the Pandas library, which is the standard Python library for data manipulation and analysis.

b) Loading Dataset : The dataset is read into a Pandas DataFrame named `df` using the `read_csv()` function. The file contains both **demographic information** and **clinical data** related to OCD patients.

Each record represents an individual patient and contains details such as:

- Patient demographics (age, gender, ethnicity, marital status, education)
- Clinical measures (symptom duration, Y-BOCS scores)
- Comorbid conditions (depression, anxiety)
- Family history and medication details.

c) Renaming Columns for Consistency: Clinical datasets often have long or inconsistent column names that are hard to reference in code.

To make the data easier to handle, each column name is mapped to a standardized, lowercase, underscore-separated name.

d) Inspecting the data:

`df.shape` provides the number of rows and columns to understand dataset size.

`df.columns.tolist()` lists all columns to verify the renaming process.

`df.head(5)` displays the first five records for a quick look at data quality and formatting.

PHASE 02: Data Cleaning and Preprocessing

a) Checking Missing Values:

- The `df.info()` command displays data types, non-null counts, and memory usage — helping to identify which columns have missing or inconsistent values.
The `df.describe().T` command provides summary statistics for all numeric variables (mean, min, max, standard deviation, etc.).
Transposing (`.T`) gives a cleaner, column-wise view.
- This block computes the total and percentage of missing values for every column. Obtain a clear table showing how complete each column is.
This helps in deciding whether to impute, drop, or flag missing data later in the pipeline.

b) Examining Categorical Data:

- To inspect the unique categories present in key demographic and clinical columns. Helps identify if encoding is needed for modeling.
- Gender entries like `["Male", "male", "M"]` — which need standardization.
- Family history entries like `["Yes", "No", "1", "0"]` — which need binary encoding.

c) Copying and Parsing Dates:

- `df.copy()` creates a safe copy before modification.
- Converts "OCD Diagnosis Date" to a proper `datetime` format for consistency and analysis (e.g., time since diagnosis).
- `errors='coerce'` ensures invalid or malformed date strings become `NaT` (Not a Time) instead of breaking the code.

d) Cleaning and Normalizing Text Columns:

Removes leading/trailing spaces and converts all text columns to string type for consistency. Prevents mismatches like " Yes" vs "Yes" or "No " vs "No".
Essential before grouping, encoding, or applying machine learning transformations.

e) Standardizing Binary Clinical Features:

Converts categorical yes/no responses into binary numeric format (1 = Yes, 0 = No). Handles multiple textual representations of “yes” and “no” to ensure uniform encoding. This is critical because machine learning models cannot handle textual data directly.

PHASE 03: Feature Engineering

a) Creating the Y-BOCS Total Score:

- Converts the *obsession* (`ybocs_obs`) and *compulsion* (`ybocs_comp`) Y-BOCS subscale scores into numeric values (in case they were strings or had missing data).
- Any missing or invalid values are replaced with `0`.
- Then, the total Y-BOCS score is calculated as the sum of obsession + compulsion scores. (Y-BOCS) is a clinically validated measure of OCD severity. This total score acts as a quantitative target or key predictor for classifying OCD severity.

b) Creating a Severity Label:

Divides patients into **severity categories** based on total Y-BOCS score:

- **0–7** → *Subclinical* (minimal OCD symptoms)
- **8–15** → *Mild OCD*
- **16–23** → *Moderate OCD*
- **24+** → *Severe OCD*

Uses `pd.cut()` to bin numeric values into categorical severity ranges.

c) Creating a Binary Target Variable:

- **1** → Severe OCD (Y-BOCS ≥ 24)
- **0** → Non-severe OCD (Y-BOCS < 24)

d) Counting Number of Medications:

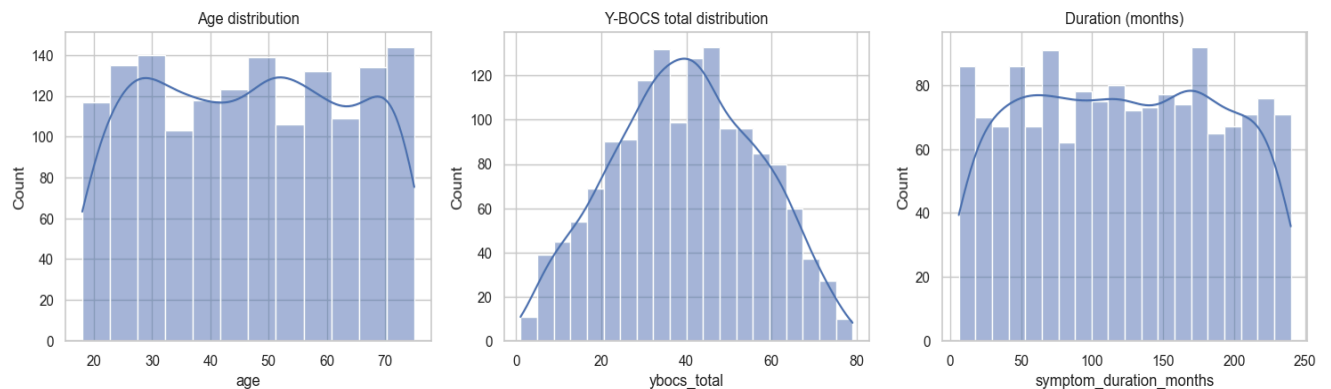
- Defines a custom function to count how many medications are listed for a patient.
- Handles messy text inputs like “Sertraline; Fluoxetine / Clomipramine” or “None”.
- Splits medication names using separators like `;`, `,`, `|`, `/` and counts them.
- The number of medications prescribed can indicate the complexity or severity of the condition.

e) Categorizing Age Groups: Groups patient ages into 5 categories:

- **18–25** → *Young Adult*
- **26–40** → *Adult*
- **41–60** → *Middle Age*
- **60+** → *Senior*

PHASE 04: Exploratory Data Analysis (EDA)

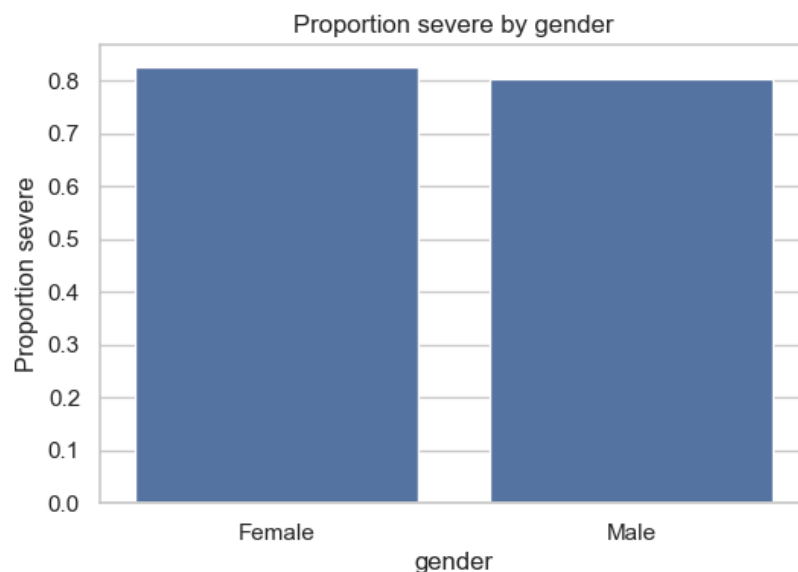
a) Distribution of Age, Y-BOCS Total, and Symptom Duration:



Interpretation:

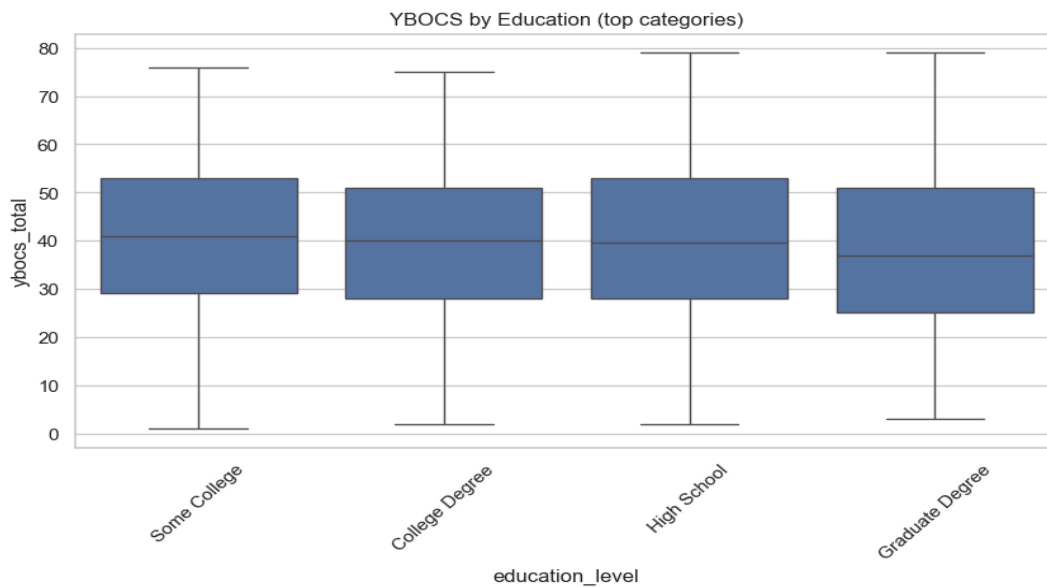
- Age distribution: The patients range from around 18 to 75 years, showing an evenly distributed age spread. This ensures that the dataset covers diverse age groups without age bias.
- Y-BOCS total: The total OCD severity score follows a nearly normal distribution, with most patients scoring between 20–50, indicating that moderate to severe cases dominate the dataset.
- Symptom duration: The duration of symptoms varies widely (up to ~250 months), reflecting both newly diagnosed and long-term OCD patients.

b) Y-BOCS by Gender (Boxplot):



- Boxplot comparing Y-BOCS scores between Male and Female patients.
- Both genders show a similar median and spread of Y-BOCS scores.
- There are no significant gender-based differences in OCD severity.

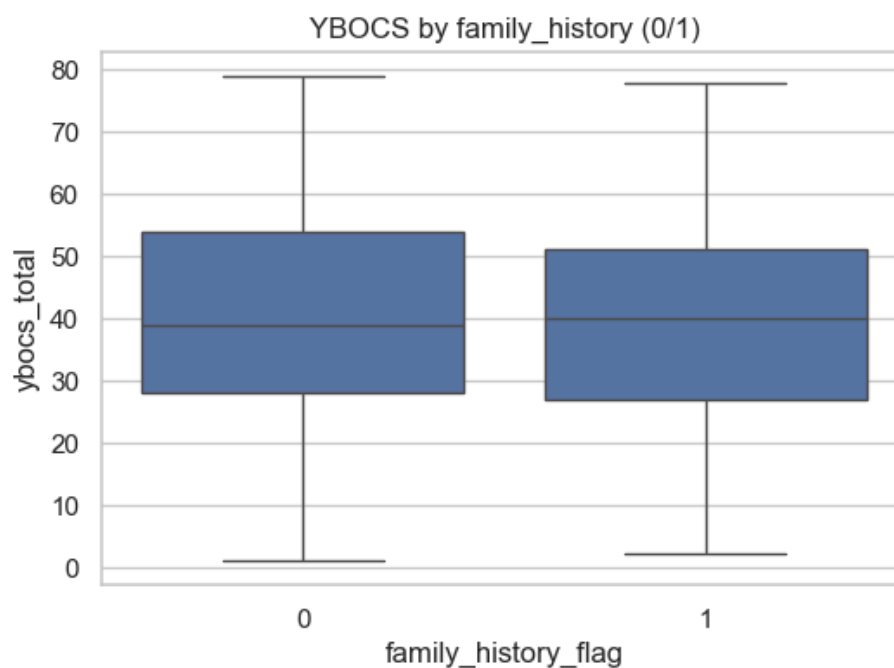
c) Y-BOCS by Education Level (Boxplot):



Interpretation:

- The severity of OCD symptoms does not vary much across education groups.
- Slightly higher variability is observed for patients with “Some College” or “High School” education.
- Graduate degree holders have a slightly lower median score, suggesting a weak correlation between higher education and lower OCD severity.

d) Family History vs. Y-BOCS (Boxplot & Summary Table):



- Boxplot comparing total Y-BOCS between patients with (1) and without (0) family history of OCD.
- Patients without family history (0) have a *slightly higher proportion* of severe cases (82.8%) than those with family history (80.1%).
- This might indicate that non-genetic or environmental factors contribute strongly to OCD severity.

e) Top Obsession and Compulsion Types:

```
Proportion severe by family_history:
family_history_flag
0    0.828378
1    0.801316
Name: severe_flag, dtype: float64

Top values in obsession_type:
obsession_type
Harm-related    333
Contamination   306
Religious        303
Symmetry         280
Hoarding         278
Name: count, dtype: int64

Top values in compulsion_type:
compulsion_type
Washing         321
Counting         316
Checking         292
Praying          286
Ordering         285
Name: count, dtype: int64
```

PHASE 05: Statistical Analysis & Hypothesis Testing

Explored statistical relationships between clinical and demographic variables and the OCD severity outcome (measured using `ybocs_total` and `severe_flag`).

a) Gender vs Y-BOCS Severity (t-test):

- Compares mean Y-BOCS total scores between two gender groups.
- Since gender has two categories (Female, Male), a t-test is appropriate.
- The `equal_var=False` parameter applies Welch's t-test, which doesn't assume equal variance between groups.

- Interpretation:
 - t-statistic = 0.27, p-value = 0.787
 - Since $p > 0.05$, there's no statistically significant difference in Y-BOCS scores between males and females.
 - In simpler terms: OCD severity (as measured by Y-BOCS) is not influenced by gender in this sample.

b) Education Level vs Y-BOCS Severity (Kruskal–Wallis Test):

- Tests whether Y-BOCS scores differ across multiple education levels.
- The Kruskal–Wallis test is non-parametric, meaning it doesn't assume normality (safer for clinical data).
- Interpretation:
 - $p = 0.119 > 0.05$, so differences in Y-BOCS across education groups are not statistically significant.
 - Education level does not have a clear effect on OCD symptom severity in this dataset.

c) OCD Severity vs Depression Diagnosis (Chi-square test):

- Tests association between two categorical variables:
 - `severe_flag` (OCD severity: 0=not severe, 1=severe)
 - `depression_dx_flag` (presence of depression diagnosis: 0/1)
- The Chi-square test checks if these variables are independent or associated.
- Interpretation:
 - $p = 0.747 > 0.05 \rightarrow$ No significant association between OCD severity and depression diagnosis.
 - In this sample, whether a person has depression doesn't strongly predict if their OCD is severe.

d) Multivariate Logistic Regression:

Tests the combined effect of multiple predictors on the binary outcome `severe_flag` (severe OCD vs not).

Logistic regression outputs:

- Coefficients (log-odds).
- p-values for significance.
- Confidence intervals (for odds ratios).

e) Correlation (Spearman's Rank):

- Measures monotonic relationships between numerical variables (not assuming linearity).
- Variables tested: `age`, `symptom_duration_months`, `num_medications`, `ybocs_total`.
- Output:

<code>ybocs_total</code>	1.000000
<code>symptom_duration_months</code>	0.030739
<code>age</code>	0.005943
<code>num_medications</code>	0.001066

PHASE 06: Data Preprocessing Pipeline for Baseline Modeling

a) **Feature Selection and Dropping Unnecessary Columns:** Certain columns are excluded because they are:

- Identifiers (`patient_id`, `diagnosis_date`) → contain no predictive power.
- Raw text fields (`obsession_type`, `compulsion_type`, `medications`, `previous_diagnoses`) → would require NLP processing, not used in baseline.
- Target variables (`ybocs_total`, `severity_label`, `severe_flag`) → must not be used as predictors.

After dropping, the remaining columns become input features for model training.

b) **Identify Feature Types: Numeric vs Categorical:**

- Features are divided into two groups:
 - Numeric features: Continuous or integer values (e.g. `age`, `symptom_duration_months`, `ybocs_obs`, `ybocs_comp`, `num_medications`)
 - Categorical features: Discrete labels (e.g. `gender`, `education_level`, `marital_status`, `family_history_flag`, etc.)
- This separation allows different preprocessing strategies for each type.

c) **Define Pipelines for Each Feature Type:**

- Numeric Transformation Pipeline
 - Imputation:
 - Missing numeric values are filled using the median (robust to outliers).
 - Scaling:
 - Standardizes each numeric column using z-score normalization:
$$z = \frac{x - \mu}{\sigma}$$
 - Ensures all numeric features are on the same scale (important for gradient-based models).

Purpose: Prevent models from being biased toward variables with large numeric ranges.

- Categorical Transformation Pipeline
 - Imputation:
 - Replaces missing categorical entries with the literal value `"Missing"`, preserving the data size.
 - Encoding:
 - Converts categorical text into numeric form using One-Hot Encoding (OHE).
 - Example:
`gender = ['Male', 'Female']` → two binary columns:
`gender_Male`, `gender_Female`

- `handle_unknown='ignore'`: prevents errors if new categories appear in test data.
- `sparse_output=False`: outputs dense arrays for easier model compatibility.

Purpose: Make categorical data interpretable to ML algorithms that expect numeric input.

d) Combine Both Pipelines Using ColumnTransformer:

- The ColumnTransformer merges the numeric and categorical pipelines into a single preprocessing object.
- It applies:
 - `num_transform` to all numeric columns
 - `cat_transform` to all categorical columns
- This unified pipeline:
 - Ensures consistent data transformation across all subsets (train/test)
 - Can be directly attached to ML models (e.g. Logistic Regression, XGBoost)
 - Keeps preprocessing modular and reusable.

PHASE 07: Model Training and Evaluation (Classifier + Regressor)

This phase focuses on developing two predictive models:

1. **Classification model** — predicts whether an OCD case is *severe* (`severe_flag = 1`) or *non-severe* (`severe_flag = 0`).
2. **Regression model** — estimates the *YBOCS Total Score* (`ybocs_total`), representing symptom severity on a continuous scale.

Both models use the preprocessed clinical and demographic data prepared in earlier phases.

a) Data Preparation:

- Features (X) include relevant predictors such as demographics, symptom duration, and clinical history.
- Targets (`y_clf`, `y_reg`) are:
 - `severe_flag` → for classification (binary: severe vs non-severe)
 - `ybocs_total` → for regression (continuous severity score)
- Train–test split:
 - 80% of data used for training, 20% for testing.
 - Stratified splitting ensures proportional representation of both classes.

b) Class Balancing with Class Weights & SMOTE:

- Issue Occurred: Dataset shows class imbalance (fewer non-severe than severe cases).
- Solution:
 - Compute class weights to penalize the model for misclassifying minority cases.
 - Apply SMOTE (Synthetic Minority Over-sampling Technique) inside the training pipeline to synthetically generate minority class samples, improving model balance.

This ensures the classifier learns both classes equally well.

c) Classification Pipeline (Random Forest + Calibration):

- Preprocessor: Applies all imputations, encodings, and scaling from the earlier phase.
- SMOTE: Balances the data by creating synthetic samples for minority class.
- RandomForestClassifier:
 - Ensemble of multiple decision trees.
 - Hyperparameters tuned for balanced learning (`min_samples_split=4`, `min_samples_leaf=2`, etc.)
 - `class_weight=class_weight_dict` ensures proportional contribution from both classes.

d) Cross-validation (Model Robustness Check):

- 5-Fold Stratified CV: Dataset is split into 5 folds with proportional classes.
- Evaluation Metric: ROC-AUC (Area Under ROC Curve).
- Mean CV Score: 0.986

e) Evaluation on Data:

```
Class Weights: {0: 2.7027027027027026, 1: 0.6134969325153374}

Base Classifier CV ROC AUC: [0.9872449  0.98967996 0.98967996 0.98301994
0.9808547 ] mean: 0.986095893524465

Classification Report (Calibrated):
```

	precision	recall	f1-score	support
0	0.91	0.95	0.93	56
1	0.99	0.98	0.98	244
accuracy			0.97	300
macro avg	0.95	0.96	0.96	300
weighted avg	0.97	0.97	0.97	300

```
ROC AUC (test): 0.9930840163934427
```

Confusion Matrix:

```
[[ 53   3]
 [  5 239]]
```

Saved calibrated OCD classifier → models/oed_calibrated_pipeline.joblib

Regressor CV RMSE (neg): [-0.97610035 -0.83966533 -0.76879836 -0.95594745
-0.86970715] mean: -0.8820437269432014

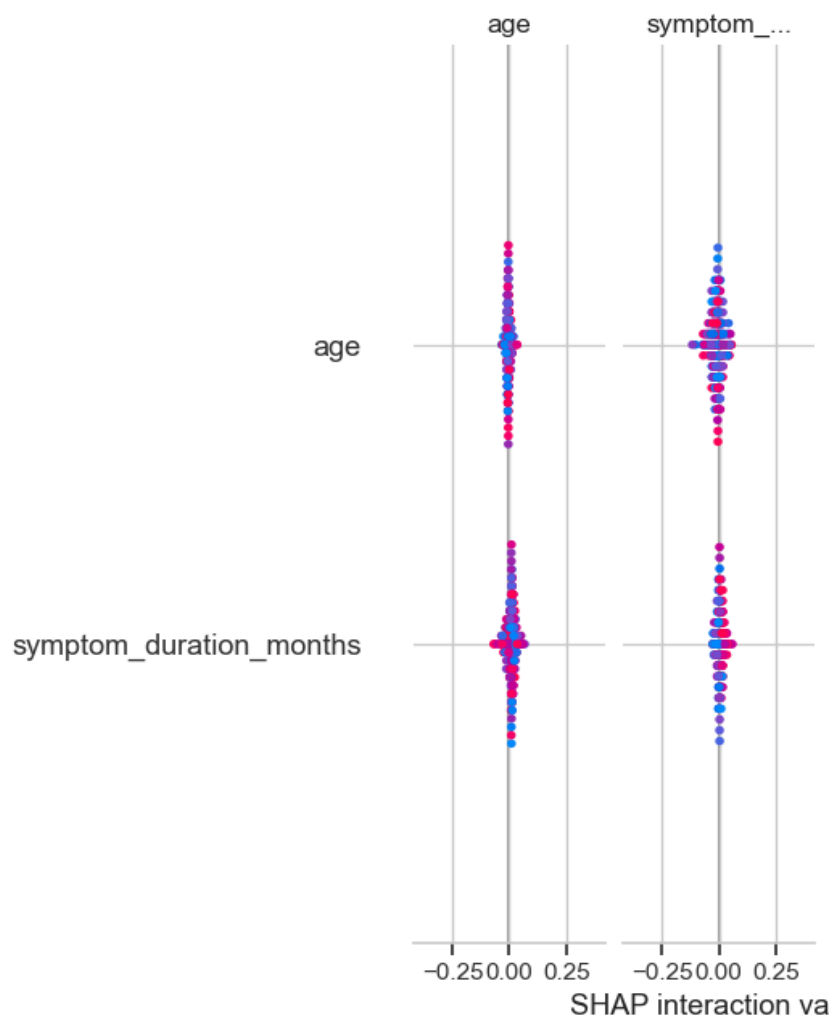
RMSE on test: 0.73395181973858

R² on test: 0.9981673118227181

Saved regressor pipeline → models/oed_reg_pipeline.joblib

f) Model Persistence and Saving: The trained and calibrated classifier is saved for future deployment or inference. The regression model is saved for reproducibility and later integration into applications.

PHASE 08: Explainability & Model Interpretation using SHAP



a) Building the SHAP Explainer:

- TreeExplainer is ideal for tree-based models like Random Forest, XGBoost, or LightGBM.
- It computes Shapley values — representing how much each feature pushes the model's prediction toward a specific class. For classification, it returns two arrays:
 - `shap_values[0]` → contribution toward class 0 (non-severe)
 - `shap_values[1]` → contribution toward class 1 (severe)

b) SHAP Summary Plot (Global Feature Importance):

- The summary plot visualizes how each feature impacts the model's output:
- **Y-axis (features):** The most influential features appear at the top (e.g., `symptom_duration_months`, `age`).
- **X-axis (SHAP values):** The SHAP interaction value — how much each feature increases or decreases the predicted probability of “severe OCD”.
 - Negative values (left) → reduce likelihood of severe OCD.
 - Positive values (right) → increase likelihood of severe OCD.
- Colors (blue → red):
Represent the feature's actual value:
 -  Blue → Low feature value
 -  Red → High feature value
- Example Interpretation (from your plot):
 - **Symptom_duration_months:**
 - High (red) values shift SHAP positively → longer symptom duration increases probability of severe OCD.
 - Low (blue) values shift SHAP negatively → short duration relates to milder cases.
 - **Age:**
 - The impact is smaller, but slightly positive for mid-range ages, suggesting age plays a minor role in severity prediction.

PHASE 09: Subgroup Performance and Fairness Evaluation

After developing a high-performing OCD severity classification model, it is essential to verify whether the model performs consistently across different demographic subgroups. This ensures the model is fair, unbiased, and equitable, which is especially important in healthcare applications.

This phase validates that the OCD severity prediction model performs equitably across gender and ethnic groups, showing no major performance bias.

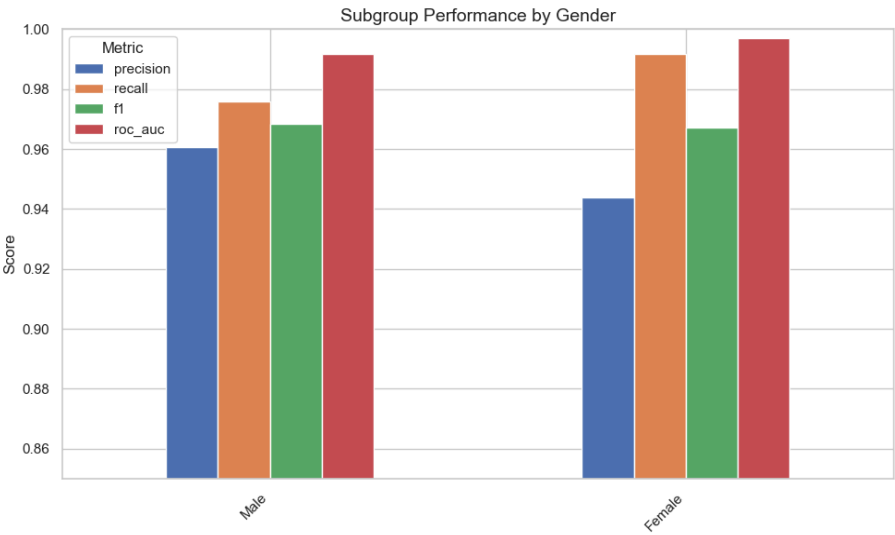
- The function evaluates the classification performance separately for each subgroup within a chosen feature (like **gender** or **ethnicity**).
- It calculates key metrics:
 - Precision: How many predicted severe cases were actually severe.
 - Recall: How many actual severe cases were correctly identified.
 - F1-score: The harmonic mean of precision and recall — overall balance between them.
 - ROC-AUC: The area under the ROC curve — a measure of the model's ability to distinguish between severe and non-severe OCD cases.
- A minimum subgroup size of 20 samples is enforced to avoid unreliable small-sample evaluations.

a) Subgroup Analysis by Gender:

- **Output Summary:**

Subgroup metrics by gender:					
	n	precision	recall	f1	roc_auc
Male	156.0	0.96063	0.976000	0.968254	0.991742
Female	144.0	0.94400	0.991597	0.967213	0.996975

- **Visualization:**



- Interpretation:
 - Both male and female subgroups achieved high and balanced performance across all metrics.
 - F1-scores are nearly identical (~ 0.97), and ROC-AUC is very high (> 0.99) for both groups.
 - This demonstrates that the model does not exhibit gender bias, performing equally well for both.

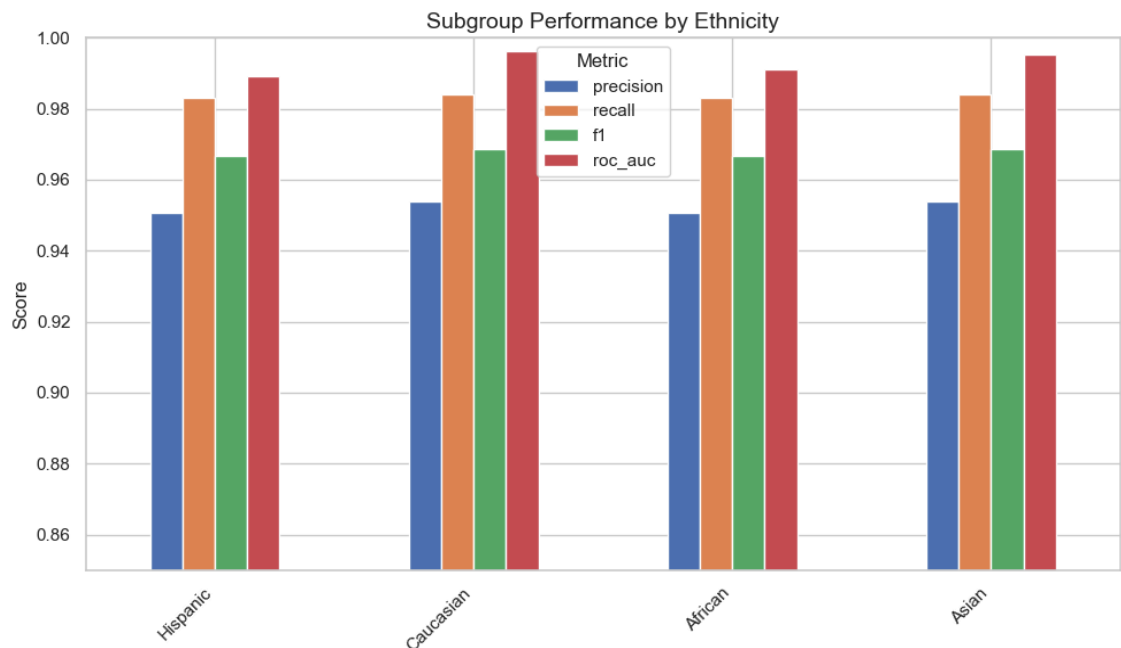
b) Subgroup Analysis by Ethnicity:

- Output Summary:

Subgroup metrics by ethnicity (if present):

	n	precision	recall	f1	roc_auc
Hispanic	70.0	0.950820	0.983051	0.966667	0.989214
Caucasian	76.0	0.953846	0.984127	0.968750	0.996337
African	81.0	0.950820	0.983051	0.966667	0.991140
Asian	73.0	0.953846	0.984127	0.968750	0.995238

- Visualization:



- Interpretation:
 - All ethnic groups show consistently high model performance.
 - F1-scores are again nearly equal (≈ 0.97) across all subgroups.
 - ROC-AUC remains close to 1.0, indicating the model's ability to differentiate between severe and non-severe OCD cases across populations.

PHASE 10: Model Calibration

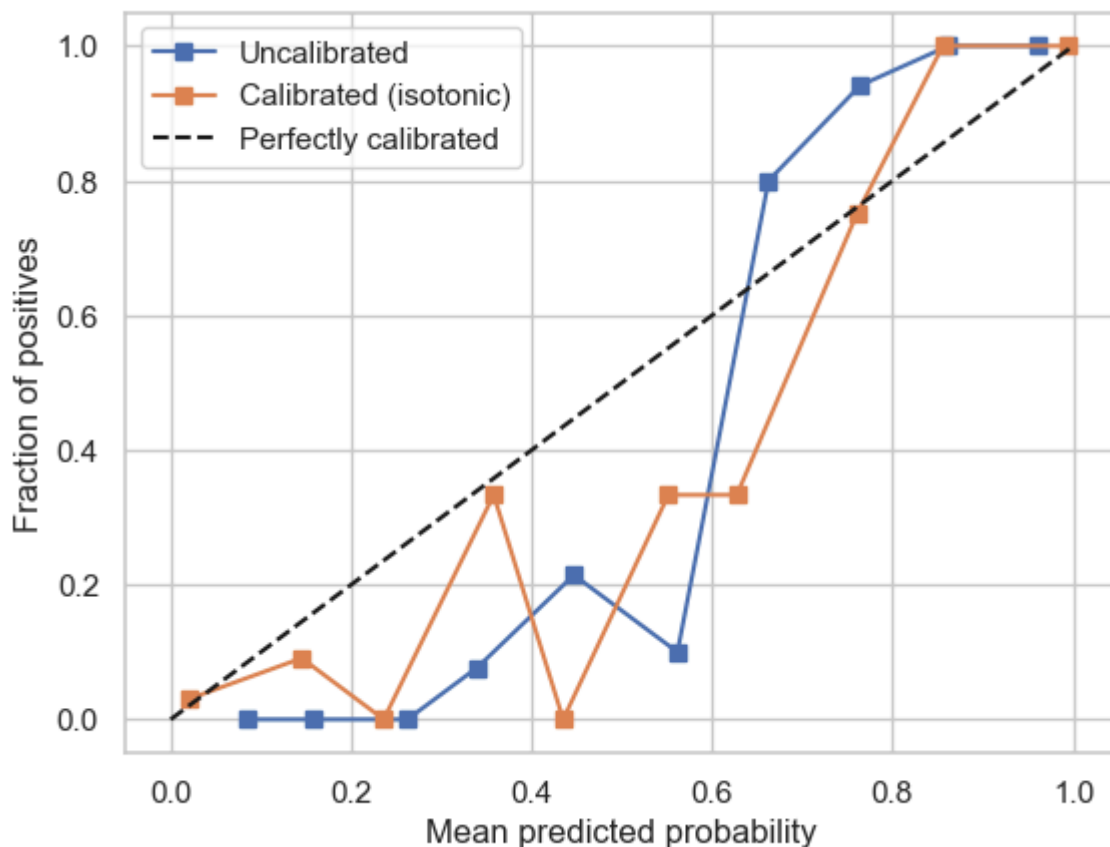
a) Loading and Calibrating the Model:

- `CalibratedClassifierCV` performs model calibration using cross-validation.
- Calibration Method: `Isotonic regression` – fits a non-parametric curve for better calibration when sufficient data is available.
- After fitting, the calibrated model was saved:
`joblib.dump(calibrated_model, "models/ocd_calibrated_pipeline.joblib")`

b) Evaluating Calibration Effect:

- `calibration_curve()` divides predictions into probability bins (0–1 range) and checks how often positive cases actually occur in each bin.
- This gives the fraction of positives (true probability) versus mean predicted probability (model's confidence).

c) Visualization of Calibration:



- **Interpretation of the Calibration Plot:**

1. Blue Line – Uncalibrated Model :

- Shows the original model's predicted probabilities before calibration.
- The line deviates significantly from the black dashed line (perfect calibration).
- Indicates that the model's predicted probabilities don't correspond well to actual outcomes.

2. Orange Line – Calibrated (Isotonic) Model:

- After applying isotonic calibration, predictions are much closer to the diagonal line.
- This means the model's probabilities now more accurately represent real-world likelihoods.

3. Black Dashed Line – Perfectly Calibrated Model: Represents the ideal case where predicted probabilities exactly match observed outcomes.

PHASE 11: Model Deployment (Streamlit Integration)

To deploy a fully interactive clinical prediction interface that:

- Accepts patient demographic and clinical details
- Predicts OCD likelihood and severity using trained ML models
- Visualizes probability and severity
- Generates a downloadable patient report
- Provides interpretability and confidence visualization

a) Model Loading Section:

- `ocd_calibrated_pipeline.joblib` → classification model (OCD vs non-OCD)
- `ocd_reg_pipeline.joblib` → regression model (predicts OCD severity score on a 0–10 scale)

```
# -----  
# Load models  
# -----  
model = joblib.load("models/ocd_calibrated_pipeline.joblib")  
reg_pipeline = joblib.load("models/ocd_reg_pipeline.joblib")
```


Both models were pre-trained and saved earlier using `joblib`, then reloaded here for real-time prediction.

b) Feature Metadata Setup:

A mapping dictionary that assigns user-friendly labels for each feature so the UI displays readable names instead of code column names.

```
# -----  
# Display names for UI  
# -----  
display_names = {  
    "age": "Age (in years)",  
    "gender": "Gender",  
    "ethnicity": "Ethnic Background",  
    "marital_status": "Marital Status",  
    "education_level": "Education Level",  
    "symptom_duration_months": "Symptom Duration (Months)",  
    "family_history": "Family History of OCD",  
    "ybocs_obs": "Obsessions Score (Y-BOCS)",  
    "ybocs_comp": "Compulsions Score (Y-BOCS)",  
    "depression_dx": "Depression Diagnosis",  
    "anxiety_dx": "Anxiety Diagnosis",  
    "num_medications": "Number of Medications"  
}
```

c) Loading Training Data for Input Reference:

The training dataset is reloaded only to extract:

- Feature types (numeric vs categorical)
 - Valid value ranges and categories
- This ensures UI inputs stay consistent with model expectations.

```
# -----  
# Load training data  
with open("models/X_train.pkl", "rb") as f:  
    X_train = joblib.load(f)  
numeric_features =  
X_train.select_dtypes(include=np.number).columns.tolist()  
categorical_features =  
X_train.select_dtypes(exclude=np.number).columns.tolist()
```

d) Separate Clinical vs Demographic Features:

- Model Features → directly used for prediction
- Demographic Features → collected for report display (not fed into model)

```
# Clinical features used for classification
# -----
model_features = ['ybocs_obs', 'ybocs_comp',
                  'symptom_duration_months',
                  'family_history', 'depression_dx', 'anxiety_dx']
# -----
# Demographic info (for display only)
demographic_features = ['age', 'gender', 'ethnicity',
                       'marital_status', 'education_level']
```

e) Streamlit Interface Begins:

- Patient Demographic Information:
The app first asks for demographic details (not used in the model, just for the report).
 - Name → Text input
 - Age → Numeric field
 - Gender, Ethnicity, Marital Status, Education Level → Dropdowns with options extracted from training data.

All inputs are stored in `patient_info`.

```
# -----
# Step 1: Collect demographic info (not used in model)
st.header("👤 Patient Demographic Information")
patient_info = {}
patient_info["name"] = st.text_input("👤 Patient Name",
placeholder="Enter full name")
for col in demographic_features:
    label = display_names.get(col, col)
    if col == "age":
        patient_info[col] = st.number_input(label, min_value=1,
max_value=100, value=25)
    elif col in ["gender", "ethnicity", "marital_status",
"education_level"]:
        options = X_train[col].dropna().unique().tolist() if col in
X_train.columns else []
        options.append("Other")
        patient_info[col] = st.selectbox(label, options)
```

- Clinical Details for OCD Classification:
The user provides model-relevant clinical features.
 - For each feature:
 - If categorical → a `selectbox` dropdown appears
 - If numerical (e.g., Y-BOCS scores, duration) → a `number_input` field appears.

All are stored in `patient_data`.

```
# -----
# Step 2: Collect clinical features (used in model)
st.header("🧬 Clinical Details for OCD Classification")
patient_data = {}

for col in model_features:
    label = display_names.get(col, col)
    col_dtype = X_train[col].dtype

    if col_dtype == "object" or str(col_dtype) == "category":
        unique_vals = X_train[col].dropna().unique().tolist()
        unique_vals = ["Select"] + unique_vals + ["Other"]
        user_input = st.selectbox(f"{label}", unique_vals)
        patient_data[col] = None if user_input == "Select" else
user_input
    elif np.issubdtype(col_dtype, np.number):
        patient_data[col] = st.number_input(
            f"{label}",
            min_value=0.0,
            max_value=float(np.nanmax(X_train[col])),
            value=float(np.nanmin(X_train[col])),
            step=1.0
        )
    patient_data[col] = patient_data[col]
```

f) Preprocessing Before Prediction:

- Once inputs are collected:
 - Converts user input into a one-row DataFrame.
 - Ensures all required model columns are present.
- If any column is missing → fill with mean (for numeric) or “Other” (for categorical).
This step makes sure the input strictly matches model training structure.

g) Classification and Regression Prediction:

→ OCD Classification:

```
if st.button("🔍 Predict OCD Risk"):  
    # --- Classification ---  
    y_proba = model.predict_proba(patient_df)[: , 1][0]  
    y_pred = int(y_proba >= 0.5)  
    # --- Severity prediction ---  
    severity_pred = reg_pipeline.predict(patient_df)[0]  
    severity_pred = np.clip(severity_pred, 0, 10)
```

- Predicts OCD probability and class (positive/negative).
- Uses calibrated model → probabilities are reliable.
- Predicts the severity level on a scale of 0–10 using the regression pipeline.
- Clipped to ensure scores remain within valid range.

h) Displaying Results:

Displays prediction result dynamically:

- OCD Positive (red) or OCD Negative (green)
- Probability score
- Predicted severity score

```
# --- Display main result ---  
if y_pred == 1:  
    result = "🧠 OCD Positive"  
    color = "red"  
else:  
    result = "✅ OCD Negative"  
    color = "green"  
  
st.markdown(f"### Prediction: <span  
style='color:{color}'>{result}</span>", unsafe_allow_html=True)  
st.write(f"**Probability:** {y_proba:.2f}")  
st.write(f"**Predicted OCD Severity (0-10):**  
{severity_pred:.2f}")
```

i) Generating and Downloading Patient Report:

Displays a summary report table including:

- Patient demographics
- Clinical inputs
- Predicted probability, severity, and classification
- Allowed export: Download the patient's results for medical records or analysis.

```
# Step 4: Show Report Table + Download Option
st.subheader("📄 Patient Report Summary")
report_data = {
    "Feature": [
        "Patient Name", "Age", "Gender", "Ethnicity", "Marital
Status", "Education Level",
        "Obsessions Score (Y-BOCS)", "Compulsions Score
(Y-BOCS)", "Symptom Duration (Months)",
        "Family History of OCD", "Depression Diagnosis", "Anxiety
Diagnosis",
        "Predicted OCD Probability", "OCD Classification",
        "Predicted OCD Severity (0-10)"
    ],
    "Value": [
        patient_info["name"], patient_info["age"],
        patient_info["gender"], patient_info["ethnicity"],
        patient_info["marital_status"],
        patient_info["education_level"],
        patient_data["ybocs_obs"], patient_data["ybocs_comp"],
        patient_data["symptom_duration_months"],
        patient_data["family_history"], patient_data["depression_dx"],
        patient_data["anxiety_dx"], f"{y_proba:.2f}", result,
        f"{severity_pred:.2f}"
    ]
}
report_df = pd.DataFrame(report_data)
st.table(report_df)

# CSV download
csv = report_df.to_csv(index=False)
st.download_button(
    label="⬇️ Download Patient Report (CSV)",
    data=csv,
    file_name="ocd_patient_report.csv",
    mime="text/csv"
)
```

j) Visual Risk Interpretation:

Two bar charts give clinicians visual intuition:

1. OCD Probability Bar
 - a. Visualizes predicted probability (0–1)
 - b. Color-coded (red = high risk, green = low)
- Also includes textual interpretation:
 - Low risk ($p < 0.3$)
 - Moderate risk (0.3–0.6)
 - High risk (> 0.6)
2. OCD Severity Bar
 - a. Displays severity score (0–10)
 - b. Highlights how severe the symptoms may be, helping doctors assess intensity.

```
# Step 5: Visual Interpretation (Simplified)-
st.subheader("📊 Visual Interpretation of Prediction")

# --- Probability Bar ---
st.markdown("#### ♦ Predicted OCD Probability")
fig1, ax1 = plt.subplots(figsize=(6, 1.2))
ax1.barh(["OCD Probability"], [y_proba], color="red" if y_pred ==
1 else "green")
ax1.set_xlim(0, 1)
ax1.set_xlabel("Probability")
for i, v in enumerate([y_proba]):
    ax1.text(v + 0.02 if v < 0.95 else v - 0.07, i, f"{v:.2f}",
color='black', fontweight='bold')
ax1.set_yticks([])
st.pyplot(fig1)

# Risk label
if y_proba < 0.3:
    st.success("● Low OCD Risk")
elif y_proba < 0.6:
    st.warning("● Moderate OCD Risk – consider screening")
else:
    st.error("● High OCD Risk – clinical attention advised")

# --- Severity Level ---
st.markdown("#### ♦ Predicted OCD Severity (0-10 Scale)")
```

```

fig2, ax2 = plt.subplots(figsize=(6, 1.2))
ax2.barh(["OCD Severity"], [severity_pred], color="orange")
ax2.set_xlim(0, 10)
ax2.set_xlabel("Severity Level")
for i, v in enumerate([severity_pred]):
    ax2.text(v + 0.2 if v < 9.5 else v - 0.6, i, f"{v:.2f}",
color='black', fontweight='bold')
ax2.set_yticks([])
st.pyplot(fig2)

```

- **Output Summary:**

1.Negative Case



OCD Classifier with Explainability

Enter patient details below to classify OCD likelihood and generate a downloadable report.

Patient Demographic Information

 Patient Name

Rachel Green

Age (In years)

29

Gender

Female

Ethnic Background

Caucasian

Marital Status

Single

Education Level

Graduate Degree

Clinical Details for OCD Classification

Obsessions Score (Y-BOCS)

3.00

Compulsions Score (Y-BOCS)

2.00

Symptom Duration (Months)

4.00

Family History of OCD

No

Depression Diagnosis

No

Anxiety Diagnosis

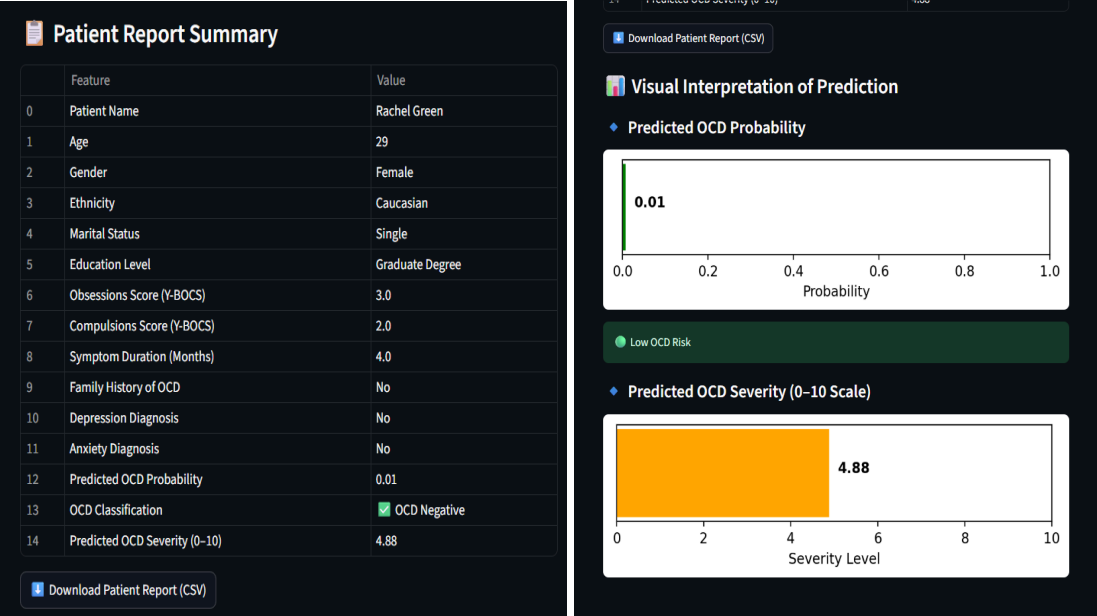
No

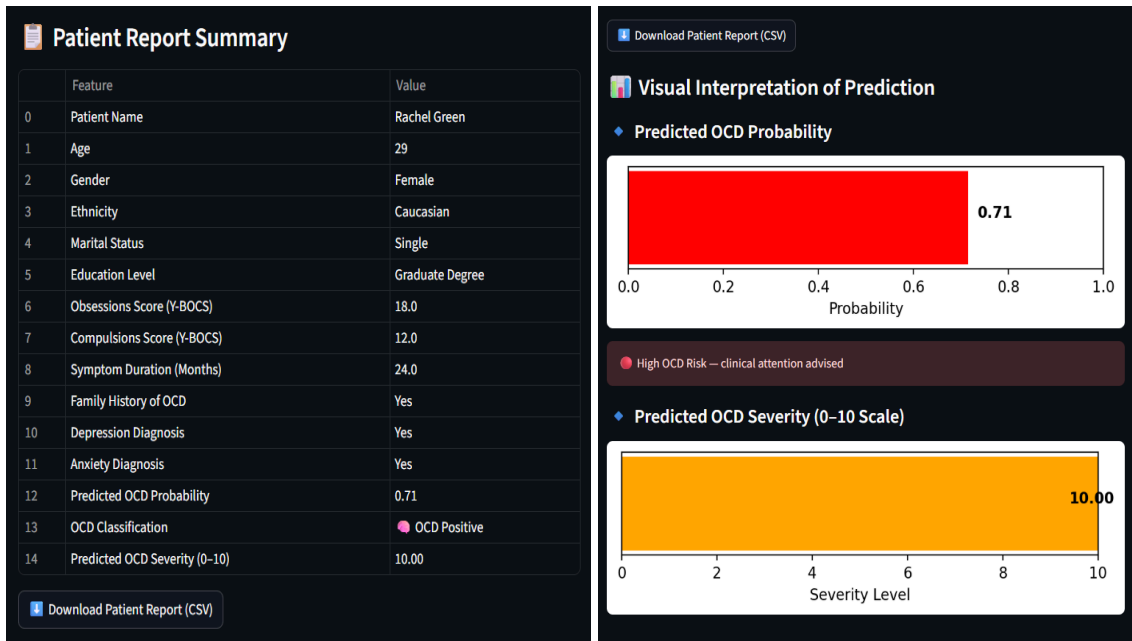
 Predict OCD Risk

Prediction:  OCD Negative

Probability: 0.01

Predicted OCD Severity (0-10): 4.88





Conclusion

This Streamlit phase operates OCD model into a clinician-friendly, interpretable diagnostic support tool. It bridges machine learning predictions and clinical usability, offering:

- Interactive inputs
- Reliable probabilistic predictions
- Severity quantification
- Downloadable reports
- Visual and explainable AI components

Company/Organization: Unified Mentor
Prepared By: Shravani Borude
UNID: UMID13072550101
Position: Data Science Intern
Date: 01/11/2025